

Dynamic Anchor: Multi-agent cooperative rule learning for class-level explanations of deep classifiers

Supreeth Suresh¹ and Suresh Muknahallipatna¹

¹Department of Electrical Engineering and Computer Science, University of Wyoming, 1000E. University Avenue, Laramie, 82071, WY, USA.

Contributing authors: ssuresh@uwyo.edu; sureshm@uwyo.edu;

Abstract

Explainable Artificial Intelligence (XAI) provides insights into how black-box models make decisions. Among the existing approaches, anchors offer high-precision human-interpretable rules in the form of simple **if-then** conditions over input features. Classical anchors compute discrete instance-wise rules using a bandit-guided beam search without learning across instances or coordinating rules across classes. Consequently, they are fundamentally local and do not yield a coherent picture of the decision regions of the model used. We propose *Dynamic Anchors*, a reinforcement learning (RL) formulation of anchor discovery, in which a policy learns to refine an axis-aligned box around an instance through a sequence of continuous actions, directly optimizing interpretable quantities such as precision and coverage. We then extend this framework to *Dynamic Multi-agent Anchors*, a cooperative game with one or more agents per class, where agents jointly learn class-wise anchor regions under shared rewards that encourage both local fidelity and a global structure, operating under defined equilibrium conditions. The trained policies were rolled out on data samples to generate both instance- and class-level rules, which were then tested globally against all classes. Experiments on standard tabular datasets showed that, first, a single-agent dynamic anchor matches or exceeds classical anchors while producing reusable policies; and second, multi-agent training yields class-wise rules with high precision, useful coverage, and reduced cross-class overlap, thereby providing a more global and structured explanation of the classifier.

Keywords: Explainable AI, Interpretability, Reinforcement Learning, Multi Agent Reinforcement Learning

1 Introduction

As machine learning models, particularly Deep Neural Networks and Gradient Boosted Decision Trees, achieve state-of-the-art performance on tabular data, their lack of transparency warrants the development of post-hoc Explainable AI (XAI) methods [1]. XAI methods aim to make black-box models more transparent by providing human-understandable descriptions of model behavior. Although feature attribution methods, such as SHAP [2] and LIME [3] identify feature influence on a prediction, they do not explicitly state the sufficient conditions required for a specific outcome. For classification tasks, *anchors* [4] are a prominent approach for local interpretability, where decision boundaries are critical. They define local, high-precision rules (e.g., **if Age > 40 and Income < 50k, then Risk = High**) that anchor a prediction in a region where the model’s behavior remains stable. Anchors are appealing because they use simple *if-then* decision rules and explicitly expose the features that matter and their numerical ranges. They provide a level of actionable data that feature importance scores alone cannot provide. However, classical anchors have certain limitations that hinder their scalability and coherence. First, the anchor computation is framed as a per-instance discrete search over the possible feature conditions. The algorithm solves a combinatorial optimization problem from scratch for each instance by generating a perturbed dataset around the instance without learning from the instances. It typically requires rigid discretization of continuous features into bins, which can introduce unnecessary boundaries and limit the resolution. In addition, the anchor aims to obtain the simplest sufficient rule to simplify the interpretability, thus limiting the number of features in forming a rule. Second, the anchors are optimized independently for each example. Although generating rules for multiple samples can be stitched together into a broader explanation, they do not yield a true class-level simple rule, and there is no mechanism to coordinate explanations across classes or ensure that the set of all anchors forms a coherent global picture of the decision regions of the classifier. In this study, we address these limitations by proposing *Dynamic Anchors*, a framework that reformulates anchor discovery as a learning problem rather than a search problem. This study makes the following contributions.

1. **Sequential Decision-Making Formulation:** We move from static, bandit-style search to a sequential decision-making process by reformulating the anchor discovery as a Markov Decision Process (MDP). This treats rule construction as a parameterized, continuous control problem, allowing an agent to iteratively refine axis-aligned hyperrectangles around the target instance or class. This approach enables high-resolution rules without the need for manual discretization.
2. The policy is trained with the RL actor-critic method to maximize the reward based on the hyperparameter set by the user, such as the precision, and coverage threshold. The RL training yields a *reusable* dynamic anchor policy that generalizes across all instances. The reformulation also allows us to extract simple rules for every class.
3. **Multi-Agent Cooperation for Global Coherence:** We extend this framework to *Dynamic Multi-Agent Anchors*, a cooperative multi-agent game with one or more agent per class and coordinate the rule set across classes. Unlike independent

optimization, these agents coordinate to learn class-specific regions within a shared reward structure. In this formulation, the agents act in parallel to refine class-specific anchor boxes to balance the local fidelity and global objectives, such as a low inter-class rule overlap and balanced coverage. We define the Nash equilibrium for this game to monitor the coordination.

We validate our framework on several standard tabular datasets. Our results demonstrate that Dynamic Anchors achieves a precision comparable to classical anchors while significantly improving coverage. Furthermore, our multi-agent approach effectively coordinates class boundaries, reducing inter-class rule overlap by a significant margin compared to the independent generation. This yields a set of decision rules that are not only locally accurate but also globally coherent.

The remainder of this paper is organized as follows: Section 2 presents related works, and Section 3 presents a brief review of traditional anchors, Reinforcement Learning (RL), and multi-agent Reinforcement Learning (MARL). Section 4 describes the base Markov Decision Process (MDP) formulation for dynamic rule generation. Section 5 expands this to a multi-agent setting for class-level coordination. Section 7.3 presents our experimental results and analysis for independent and coordinated multiagent anchors. Finally, Section 9 presents the conclusions and future work.

2 Related Work

The study of interpretable “if-then” rules has evolved considerably. Early methods, such as CN2 [5] and RIPPER [6], used heuristic and greedy searches to construct rules. Although this is effective for simple tasks, these greedy approaches often lack the flexibility to approximate the complex decision boundaries found in modern deep neural networks. More recently, local rule-based methods, such as Anchors [4] have emerged as powerful alternatives. Anchors optimize the rules for individual instances, providing high-precision sufficient conditions for specific predictions. However, the original Anchors algorithm relies on a multi-armed bandit formulation that requires solving a combinatorial search from scratch for every new instance, thereby preventing knowledge transfer or amortized inference. More recent approaches have attempted to learn rules using gradient-based optimization. Methods such as Neural Decision Trees [7, 8] relax strict logical thresholds into soft, differentiable functions to enable end-to-end training. However, these “soft” rules often lack true interpretability. Converting them back into crisp, “hard” logic rules for human analysis typically degrades their precision and performance. Another popular method involves using reinforcement learning to structure the learning process. Roth et al. [9] train agents to sequentially build decision trees; VIPER [10] uses imitation learning to distill neural networks into tree structures. However, these methods still treat rule construction as a discrete sequence of split decisions. In contrast, our framework treats rule discovery as a continuous-control problem. By allowing agents to dynamically resize decision boundaries in a continuous space, our method avoids rigid discretization and efficiently generates anchors with high resolution.

3 Background

3.1 Anchors for local explanations

Anchors explain the individual predictions of a black-box model $f(\theta)$ by constructing local and high-precision rules. For a given sample instance $x \in \mathbb{R}^d$ with prediction $c = \arg \max_k f_k(x)$, an anchor is a set of minimal feature conditions $A(x)$ such that data perturbations in a small local region are consistent with the anchor, and the model prediction inside that region is stable. Anchors usually has the form

$$A(x) \equiv \forall_{j \in S} (\ell_j \leq x_j \leq u_j) \quad (1)$$

where $S \subseteq \{1, \dots, d\}$ are the features and (ℓ_j, u_j) are the anchor bounds of the features. Anchors are optimized to maximize *precision*

$$\text{prec}(A) = \mathbb{E}_{D_x(z|A)} [1_{\hat{f}(x)=\hat{f}(z)}] \quad (2)$$

such that $\text{prec}(A) \geq \tau$ and subject to sufficient *coverage*,

$$\text{cov}(A) = \mathbb{E}_{\mathcal{D}_{(x)}} [A(z)] \quad (3)$$

under a sampling distribution around the sample x , where τ is a predefined threshold (e.g., 0.95). Traditional anchor algorithms frame this as a bandit problem over candidate conditions using a greedy beam search and Monte Carlo estimates of precision and coverage.

3.2 Reinforcement learning recap

A Markov Decision Process (MDP) provides a structured framework for an agent to learn decision-making strategies that maximize cumulative rewards. An MDP is defined by a tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where $\mathcal{S}$ and $\mathcal{A}$ are the state and action spaces, P is the transition kernel; r is the reward function; and $\gamma \in [0, 1)$ is the discount factor. At each time step t , the agent observes the state $s_t \in \mathcal{S}$, takes action $a_t \in \mathcal{A}$, receives a reward r_t , and transitions to a new state $s_{t+1} \sim P(\cdot | s_t, a_t)$. The goal is to learn a policy $\pi(a | s)$ that maximizes the expected discounted return $J(\pi) = \mathbb{E}_\pi[\sum_{t=0}^{\infty} \gamma^t r_t]$. The expected returns and policies that determine the action to be taken in a particular state can be parameterized using neural networks. The policy gradient and actor-critic methods are commonly used to optimize parameterized policies π_θ in continuous action spaces. The formulation of anchors as an MDP and the optimization of the policy are explained in Section 4. In our implementation, we used the Deep Deterministic Policy Gradient (DDPG) and Soft Actor Critic (SAC) methods with continuous control algorithms to solve the policy optimization.

3.3 Multi-agent reinforcement learning

While standard RL focuses on a single agent, Multi-Agent Reinforcement Learning (MARL) involves multiple agents interacting within a shared environment. A multi-agent extension of an MDP is often described as a stochastic game defined by a tuple $(\mathcal{S}, \{\mathcal{A}^{i=1\cdots N}\}, P, \{r^{i=1\cdots N}\}, \gamma)$, where $\mathcal{S}$ is the global state space, $\{\mathcal{A}^{i=1\cdots N}\}$ is the action space of agent i , P is the joint transition kernel, $\{r^{i=1\cdots N}\}$ is the reward function for agent i , and $\gamma \in [0, 1)$ is the discount factor. At each time step t , each agent i observes state s_t^i selects an action $a_t^i \in \mathcal{A}^i$, and the environment transitions to $s_{t+1} \sim P(\cdot | s_t, a_t^1, \dots, a_t^N)$ while each agent receives a reward $r_t^i = r^i(s_t, a_t^1, \dots, a_t^N)$.

In most cases, multiple agents can interact in three ways: cooperatively, competitively, or in a mixed manner. In a fully cooperative setting, all agents share a common objective and are typically optimized with respect to a shared expected return. This involves optimizing policies jointly defined by $\boldsymbol{\pi} = \{\pi^i(a^i | s^i)\}_{i=1}^N$ (N is the number of agents), and the goal is to learn policies that maximize the expected cumulative reward R . To achieve this, algorithms often utilize fictitious play [11] or a variant to train the joint policies. Among these variants, the Centralized Training and Decentralized Execution (CTDE) is a common design that addresses the non-stationarity inherent in multi-agent learning. In CTDE, a centralized critic has access to the joint state and actions during training, whereas each agent’s actor only uses its local observation at execution time. The formulation of dynamic multi-agent anchors as a cooperative stochastic game, along with the design of class-wise agents, shared rewards, and overlap penalties, is described in Section 5. In our implementation, we use multi-agent DDPG (MADDPG) [12] and multi-agent SAC (MASAC) [13] algorithms from the BenchMARL package [14] for training and execution. Both MADDPG and MASAC follow the CTDE paradigm.

4 Dynamic Anchors - base MDP

We first describe our base MDP formulation, in which a policy learns to refine the bounds of an axis-aligned box for a given target instance or class as follows: The main advantage of this method over traditional anchors is that it produces a reusable policy capable of estimating anchors for both individual samples and all classes.

4.1 Problem setup

Let $f : \mathbb{R}^d \rightarrow \Delta^{K-1}$ be a fixed classifier over K classes defined on inputs $x \in \mathbb{R}^d$, where x is normalized to $[0, 1]^d$. Given a specific target prediction to explain (either a specific instance x^* or a target class c), we seek a hyper-rectangle anchor $\mathbf{B}$ defined as

$$\mathbf{B} = \{x \in [0, 1]^d \mid \ell \leq x \leq u\} \quad (4)$$

$$0 \leq \ell < u \leq 1$$

where ℓ_j and u_j denote the normalized lower and upper bounds for the j -th feature, respectively. The objective is to adjust these bounds to identify a region $\mathbf{B}$ that maximizes the precision and coverage for the target, subject to validity constraints. We

formulate anchor discovery as a sequential decision-making problem. An agent interacts with the environment that represents the current box $\mathcal{B}_t = [\ell_t^d, u_t^d, p_t, c_t]$ as its current state, where d is the number of features, takes an action a to shrink or expand the bounds, and obtains a reward r based on the precision, coverage, rules validity, and penalties. The bounds are updated based on the actions to obtain the next state. This continues until we achieve both the precision and coverage thresholds, or the maximum number of steps in an episode. We used weighted versions of the thresholds to guide the learning process smoothly. The objective is to maximize the total accumulated rewards to maximize both the precision and coverage based on the threshold. This process naturally fits a *Markov Decision Process (MDP)* framework, allowing us to leverage powerful reinforcement learning to learn the optimal box refinement policies.

4.2 MDP formulation

This study focuses only on classification tasks with tabular numerical datasets. For simplicity, faster convergence, and paving the way for multiagent comparison, we trained individual policies that were unique to each class. The instance or class that we interpret is associated with an environment $\mathcal{E} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ defined as follows:

State: At timestep t , the state is

$$s_t = [\ell_t, u_t, p_t, c_t] \quad (5)$$

where $\ell_t, u_t \in [0, 1]^d$ are the normalized lower and upper bounds of the current box, p_t is the current precision; and c_t is the current coverage. The initialization of the RL bounds depends on the specific type of interpretation used. For instance-specific interpretation, we initialized a small window around the target instance. In contrast, for class-specific interpretation, we performed k -means clustering on the training data to identify k -representative clusters. To ensure strictly non-zero initial coverage, which is crucial in sparse, high-dimensional spaces, we do not arbitrarily initialize around the centroid. Instead, we identify the k nearest neighbors to the cluster centroid and define the initial box bounds to enclose these samples minimally. Finally, to prevent the box from collapsing, we generated locally perturbed datasets using different strategies based on data density: we used bootstrapping (combining perturbations and true instances) for dense samples and applied uniform perturbations for sparse samples.

Action: Unlike traditional anchors, we used a continuous action to adjust the bounds. The agent adjusts the feature bounds to refine $\mathcal{B}_t$. The Deep Deterministic Policy Gradient (DDPG) and Soft Actor Critic (SAC) were used to support continuous actions. Here, we predict a real-valued vector $(\Delta\ell_t, \Delta u_t) \in [-1, 1]^d$ scaled by the box width, which allows for a smooth dimension-wise finer control over bound updates. We use a *tanh* activation to bound the changes to between $[-1, 1]$. After applying the bounding box changes, the new values of the bounding box, along with the new precision and coverage, become the next state s_{t+1} . One caveat is that we must ensure that all data are normalized to values between $[0, 1]$.

Model transitions The environment updates the box deterministically based on the agent’s action as follows:

$$\ell_{t+1} = \ell_t + \Delta\ell_t, \quad u_{t+1} = u_t + \Delta u_t, \quad (6)$$

The agent also uses a clipping function to enforce $\ell_{t+1} < u_{t+1}$ and a minimum box width $\epsilon_{\min}$ to prevent complete collapse.

4.3 Environment

Reward: Reward design is one of the most crucial factors in designing an RL pipeline. It should support good actions and punish bad ones. Good actions that improve precision and coverage are carefully balanced by penalizing the instability and drifting. The reward function is designed to balance exploration (coverage) and exploitation (precision) while enforcing the geometric constraints. The reward at time t is defined as

$$r_t = \underbrace{\alpha\Delta P_t + \beta\Delta C_t}_{\text{Gains}} + \underbrace{R_{\text{cov}} + R_{\text{purity}}}_{\text{Bonuses}} - \eta \cdot \left(\underbrace{\lambda_d D_t + \lambda_a A_t + \lambda_g G_t + \gamma O_t}_{\text{Constraints}} \right) \quad (7)$$

where the components are defined as follows:

- **Gains:** ΔP_t and ΔC_t denote the step-wise gains in precision and coverage, weighted by α and β respectively (default $\alpha = 0.7$, $\beta = 1.0$).
- **Bonuses:** To aid convergence, we apply two distinct bonuses:
 - **Coverage Bonus (R_{cov}):** Triggered when precision thresholds are met to encourage volume expansion.
 - **Target Purity Bonus (R_{purity}):** Incentivizes capturing any instance of the target class to prevent box collapse into empty space.
- **Constraints:** Penalties are scaled by a dynamic progress factor $\eta \in \{0.5, 0.7, 1.0\}$ to balance exploration and stability. We dynamically adapt η based on agent progress: $\eta = 0.5$ during positive gains, which encourages expansion; $\eta = 0.7$ when precision is near the target ($> 0.8\tau_P$); and $\eta = 1.0$ otherwise to strictly enforce constraints. The specific penalties are as follows:
 - D_t : Trajectory drift penalty (weighted by λ_d), minimizing high-variance updates using $\|B_t - B_{t-1}\|_2$.
 - A_t : Anchor drift penalty (weighted by λ_a), keeping the box center near the initialization centroid x^* .
 - G_t : Geometric validity penalty (weighted by λ_g), which acts as a proxy for Jensen-Shannon volume stability and enforces coverage floor constraints.

- O_t : Overlap penalty (weighted by γ), which penalizes degenerate boxes with dimensions narrower than the minimum width ($< 2 \times \text{min_width}$).

4.4 Learning dynamic anchor policies

The anchor policy function $\pi_\theta(a|s)$ is learned using off-policy algorithms that are suitable for continuous control. We use the Deep Deterministic Policy Gradient (**DDPG**) and soft actor-critic (**SAC**) implementations from the Stable Baselines 3 (**sb3**) package [15]. To stabilize training and improve sample efficiency, we used vectorized environment features to parallelize the experience collection across multiple independent instances. The training dynamics are illustrated in Figure 1. The learning process typically follows three phases.

- **Exploration and Expansion:** In the early stages, the *coverage bonus* incentivizes the agent to expand the box volume rapidly. To do this efficiently without evaluating the full dataset, the environment uses perturbation sampling to estimate the precision and coverage. During this phase, if the agent achieves positive gains in either metric, the progress factor is relaxed to $\eta = 0.5$, allowing the agent to explore the feature space with looser constraints
- **Refinement:** As the anchor covers more data samples, the precision gain becomes the dominant signal. The agent learns to shrink or shift the boundaries to exclude the non-target instances. The target purity bonus and coverage floor constraint ensure that the box does not collapse into empty regions during refinement.
- **Stabilization:** Once the precision approaches the target ($> 0.8\tau_P$), the penalty scalar increases to $\eta = 0.7$ or $\eta = 1.0$ if no further gains are made. This stricter enforcement of Trajectory Drift (D_t) and Anchor Drift (A_t) dampens the policy updates, preventing the agent from oscillating around the optimal solution or forgetting the valid rule owing to high-variance updates.

Both DDPG and SAC successfully converged to deterministic continuous actions that maximized the trade-off between coverage and precision, although the adaptive η schedule proved critical for preventing mode collapse in the DDPG implementation.

5 Dynamic multi-agent anchors for coordinated class level explanations

We extend the dynamic anchor base MDP to multiple agents to introduce coordination between agents, enabling the formation of precise and coherent class-wise rules. This forms the basis for the full dataset and trained model-specific explanation. We model the construction of class-wise anchors as a cooperative multi-agent game played in a high-dimensional feature space. Intuitively, each class is represented by an agent or a group of agents that proposes a simple, human-interpretable rule of the form if the features fall inside this box, then the prediction is reliably class c . Each agent jointly adapts its boxes to achieve high precision and coverage for its class while avoiding overlap with other anchors of competing classes and minimizing redundancy within the same class.

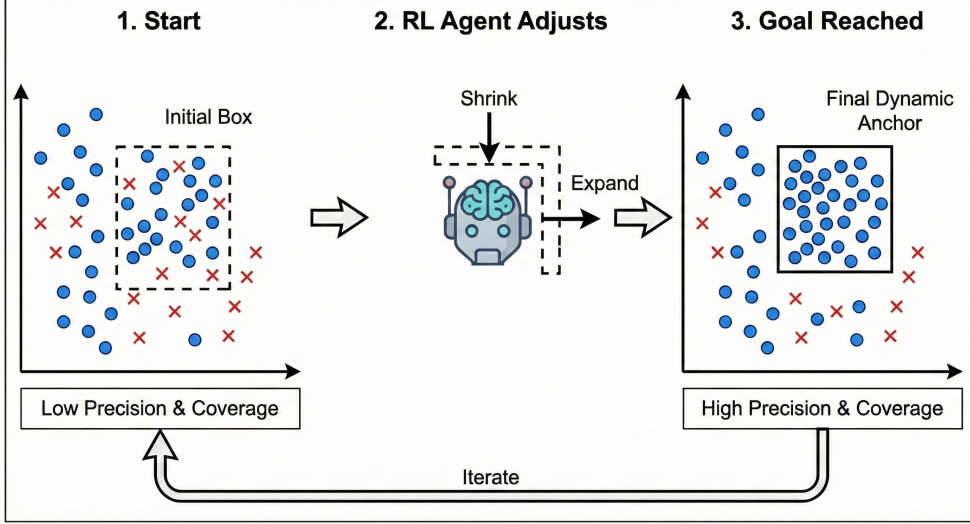


Fig. 1: Dynamic anchor training dynamics

5.1 Problem setup for anchors

Let $f : \mathbb{R}^d \rightarrow \Delta^{K-1}$ be a fixed classifier over K classes defined on inputs $x \in \mathbb{R}^d$, where x is normalized to $[0, 1]^d$. For a dataset $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$, we seek, for each class $k \in \{1, \dots, K\}$, a set of M **hyper-rectangle anchors** $\mathcal{A}_k = \{\mathbf{B}_{k,1}, \dots, \mathbf{B}_{k,M}\}$, where each anchor $\mathbf{B}_{k,i}$ is defined as:

$$\mathbf{B}_{k,i} = \{x \in [0, 1]^d \mid \ell_{k,i} \leq x \leq u_{k,i}\} \quad (8)$$

$$0 \leq \ell_{k,i} < u_{k,i} \leq 1$$

where $\ell_{k,i}, u_{k,i} \in [0, 1]^d$ are the lower and upper bound vectors for the i -th agent of class k .

Each anchor possesses the following characteristics: high precision for class k , non-trivial coverage over $\mathcal{D}$, and minimal overlap with other anchors. Original anchors are instance-based and utilize precision and coverage as metrics. However, dynamic anchors aim to provide a class-level explanation. Therefore, we introduce new metrics, namely class-level metrics and *class-union metrics*, which adequately explain the efficiency of a set of anchors. Section 6 describes the metrics in detail.

5.2 Markov game formulation

The interactions between agents form a game. We formulate this as a cooperative Markov game with boundary restrictions. Each anchor corresponds to a simple rule "if $x \in B_k$, then the model's prediction is reliably class k " and the goal of the game is to jointly adjust $\{B_k\}_{k=1}^K$ to make these rules consistent globally. Let $\mathcal{J} = \{1, \dots, K\}$ index the agents/classes.

State: The combined state $\mathbb{S}_t$ contains the configuration of all agents $B_{1,t}, \dots, B_{K,t}$, where $B_{k,t}$ is the current box of agent k . The dataset $\mathcal{D}$ and classifier f are fixed throughout the training. Each anchor is parameterized by its lower and upper bounds

$$B_{k,t} = \{x \in [0, 1]^d \mid \ell_{k,t} \leq x \leq u_{k,t}\} \quad (9)$$

The game is partially observable. Agent k receives a local observation $o_{k,t}$ consisting solely of its own box coordinates and performance metrics.

$$o_{k,t} = (\ell_{k,1:d,t}, u_{k,1:d,t}, P_{k,t}, C_{k,t}) \quad (10)$$

The agent does not observe the exact bound changes of the other agents. It only receives feedback through shared rewards and interaction penalties.

Action: At each step, agent k outputs a continuous action vector that is passed through a $\tanh$ activation to limit the output to $a_{k,t} \in [-1, 1]^{2d}$. The continuous actions from the policy form the relative adjustments to its lower and upper bounds $\Delta \ell_{k,j,t} \approx a_{k,j,t}^{(\ell)}$, $\Delta u_{k,j,t} \approx a_{k,j,t}^{(u)}$ scaled by the current box width and clipped to enforce box constraints $0 \leq \ell_{k,j,t+1} < u_{k,j,t+1} \leq 1$, $\ell_{k,j,t+1} - u_{k,j,t+1} \geq w_{min}$. We additionally impose a coverage floor constraint to avoid the complete collapse of boxes. If an update drives the coverage below the minimum threshold, we revert to the previous box and count the coverage floor violation. This keeps the anchors meaningful and prevents degenerate solutions from collapsing into tiny regions.

Model transition: Given the current state s_t and joint action $\mathbf{a}_t = (a_{1,t}, \dots, a_{K,t})$: Each agent's box is updated using its action and clipping rules. Given the new boxes, we recompute the coverage and blended precision for each class using either the full dataset or a stochastic mini-batch/perturbed-sample set. Finally, the new state s_{t+1} is obtained by replacing $\{B_{k,t}\}$ with $\{B_{k,t+1}\}$ and updating ξ_t . This induces a Markovian transition kernel $P(s_{t+1} \mid s_t, \mathbf{a}_t)$ with stochasticity only because of the sampling/perturbation procedure.

5.2.1 Local and Shared Rewards

We designed the reward such that each agent was incentivized to improve its anchor while coordinating with others to avoid any overlap. In addition, we encourage all agents to achieve global precision and coverage targets through a shared reward. The shared reward was scaled appropriately, based on the progress of the agents. We also included additional bonuses when the agents achieved partial coordination to encourage this behavior. Although the reward structure is complex, the inclusion of each term and its weight can be controlled by the user. This allows us to be flexible with settings in which the agents struggle to achieve good results. We provide the optimal settings range for each hyperparameter in Section 7.3.

The reward function in the multi-agent environment is designed to guide the agents towards finding high-precision anchors that satisfy coverage constraints while minimizing redundancy between agents. The total reward R_{total} for an agent i at time step t is composed of a local reward R_{local} which focuses on individual anchor quality,

a shared cooperative reward R_{shared} which focuses on collective performance, and a bonus reward R_{class} for overachieving the goal.

Local Rewards (R_{local}): The local reward incentivizes individual agents to optimize the precision and coverage of their specific anchor box. The local reward term is the same as that in the single-agent case, given by Equation ?? . To encourage continuous improvement, the agents are rewarded based on the change in metrics between steps (ΔP_t and ΔC_t). While the overall goal is to maximize both precision and coverage, priority is given to precision, as the trustworthiness of the anchor matters more. The weights w_P and w_C are dynamic and are described as follows

Precision Weight (w_P): Increases as precision approaches the target threshold τ_P

$$w_P = \begin{cases} \max(2.0, 1 + \frac{P_t - \tau_P}{1 - \tau_P}) & \text{if } P_t \geq \tau_P \\ 2.0 & \text{otherwise} \end{cases} \quad (11)$$

Coverage Weight (w_C): Scaled by the hyperparameter beta and the current precision, penalizing coverage gains if precision is low

$$w_C = \beta \cdot \min\left(1.0, \frac{P_t}{\tau_P + \epsilon}\right) \quad (12)$$

Shared Cooperative Reward (R_{shared}): To induce cooperative behavior, a shared reward is given to all agents. This component drives all agents to cooperatively solve the overall task. This is given by the equation

$$R_{shared}^{(t)} = w_{shared} (\mathbb{I}_{all_met} + 0.5 \cdot f_{met} + 0.3 \cdot (1 - \bar{O}) + B_{global_cov}) - P_{inter} - P_{same} \quad (13)$$

where Global Success ($\mathbb{I}_{all_met}$) is a binary indicator (1.0) if all active agents meet their precision and coverage targets, Partial Success (f_{met}) is the fraction of agents currently meeting their targets, and Overlap Avoidance ($\bar{O}$) is the average pairwise overlap ratio between all agents. A lower overlap yields a higher reward. Global Coverage (B_{global_cov}): A bonus proportional to the union coverage of all agents ($\bigcup A_i$), incentivizing the swarm to cover distinct regions of the input space. Inter-class Overlap (P_{inter}): Penalizes the volume intersection between agent i and any agent j of a different class ($c_j \neq c_i$). This minimizes the ambiguity of the global decision boundaries. Same-class Diversity (P_{same}): Penalizes the Jaccard index between agent i and any agent j of the same class ($c_j = c_i$). This forces multiple agents representing the same class to cover distinct data modes, rather than converging to the same rule.

Class union reward (R_{class}): If enabled, this component rewards the collective performance of all agents assigned to a specific class c :

$$R_{class}^{(c,t)} = w_{union_cov} \cdot C\left(\bigcup_{j \in c} A_j\right) + w_{union_prec} \cdot P\left(\bigcup_{j \in c} A_j\right) \quad (14)$$

This specifically encourages agents of the same class to form a high-quality composite rule that effectively covers the class distribution.

5.2.2 Episode Structure and Termination

The game proceeds for at most $T_{\max}$ steps. For each agent k , we terminate early (and freeze its box) if the anchor has converged. We used multiple progressively harder convergence criteria to terminate each episode. We maintain individual criteria counters to introduce priority in selecting the termination conditions. Once all agents are terminated or $T_{\max}$ is reached, the episode ends, and the final anchors $\{B_{k,T}\}_{k=1}^K$ are returned as explanations. *Termination logic:* Dynamic anchors utilizes a hierarchical system. An episode terminates if an agent satisfies any of the following conditions: i) Ideal: Precision $\geq \tau_P$ and Coverage $\geq 0.95\tau_C$. ii) High Precision: Precision $\geq \tau_P$ AND Coverage $\geq 0.7\tau_C$. iii) Precision $\geq \tau_P$ AND Coverage $\geq 0.5\tau_C$. To prevent the agents from exploiting easy termination conditions and getting stuck at the local minimum, the multiagent environment maintains counters for each termination reason. If a specific reason is used too frequently, it is temporarily disabled, forcing the agents to strive for harder targets. The counters for each termination are user-defined, and the values are controlled using an anchor environment configuration file. The default limits were set empirically to force the agents to move out of the local optima.

5.2.3 Training

We trained the setup as a parallel multi-agent reinforcement-learning (MARL) problem using a centralized training/decentralized execution (CTDE) paradigm. Specifically, we implemented our environment as a PettingZoo [16] **ParallelEnv** and interfaced it with BenchMARL/TorchRL via a thin wrapper that exposes continuous actions and per-agent observations. Each agent corresponds to a class, whereas execution remains decentralized: agent k selects actions using only its local observation.

Training proceeds in rollouts of length at most `max_cycles`. At each timestep t , we (i) collect the joint observations $\{o_{k,t}\}_{k=1}^K$; (ii) sample continuous actions $\{a_{k,t}\}$ from the decentralized actors $\pi_{\theta_k}(a_k \mid o_{k,t})$; (iii) step the environment to obtain the next joint observations, per-agent rewards, and termination signals; and (iv) store transitions for policy updates. We optimize policies using MADDPG: each class/agent maintains an actor network, whereas a centralized critic conditions on the joint information to stabilize learning in the presence of multi-agent non-stationarity. In practice, this yields stable updates, even though each agent’s reward depends on the coupled quantities.

5.3 Nash Equilibrium

In this context, each *class* is like a player in a Markov game. A joint policy $\pi = (\pi_1, \dots, \pi_K)$ creates a distribution over possible outcomes, and each player c wants to maximize its expected return $J_c(\pi_c, \pi_{-c})$. A Nash equilibrium π^* occurs when no

player can do better by changing their strategy alone:

$$\begin{aligned} J_c(\pi_c^*, \pi_{-c}^*) &\geq J_c(\pi_c, \pi_{-c}^*), \\ \forall c \in \{1, \dots, K\}, \forall \pi_c. \end{aligned} \quad (15)$$

Finding the exact best response is difficult for neural policies in continuous action spaces; therefore, we use an approximate method called *NashConv* [17]. We check whether a class can improve its estimated value by changing its action while others remain the same:

$$\Delta_c(s) = \max_{a_c} Q_c(s, a_c, a_{-c}) - Q_c(s, a_c^\pi, a_{-c}^\pi). \quad (16)$$

We calculate Δ_c for sampled states using a learned centralized critic. We approximate the best action a_c using a gradient-based ascent or random search when the gradients are unreliable. We then average these to obtain the per-class exploitability $\mathbb{E}[\Delta_c]$.

$$\begin{aligned} \text{NashConv}(\pi) &= \sum_{c=1}^K \mathbb{E}[\Delta_c], \\ \text{Exploitability}_{\max}(\pi) &= \max_c \mathbb{E}[\Delta_c]. \end{aligned} \quad (17)$$

In our tests, a joint policy is an ε -*equilibrium* if these exploitability measures are small, meaning that no class can greatly improve by changing its action alone. We also want solutions that meet the precision and coverage goals for each class and have low NashConv, showing that the class anchors are high-quality and stable. We only use the NashConv metric to monitor the efficiency of the training and not as a termination condition.

6 Evaluation Metrics

We employed a hierarchical evaluation framework to rigorously assess the interpretability and fidelity of the generated anchors. We distinguish between instance-level metrics, which evaluate the local fidelity of specific samples, and class-level metrics, which evaluate the fidelity of the rule set at the class level. Although both single- and multi-agent frameworks generate rules that can be evaluated at all levels, the multi-agent framework is explicitly designed to optimize the collective class-level performance through cooperative rewards. All instance-based metrics are computed on a held-out test set $\mathcal{D}_{test}$ to ensure that the rules generalize beyond the training distribution. On the other hand, the class-based metrics are computed on the full dataset $\mathcal{D}$ to ensure that we obtain a set of rules specific to a class.

6.1 Instance-Level Metrics

These metrics assess the quality of an individual anchor box B generated for a specific target instance, x .

Hard Precision:

Precision measures the trustworthiness of a rule. While the training phase utilizes a *soft* precision proxy blended with model probabilities to differentiate gradients, we evaluate the final rules using *hard* ground-truth labels to ensure strict fidelity:

$$\text{Precision}(B, c) = P(y = c \mid x \in B) \approx \frac{1}{|B \cap \mathcal{D}_{test}|} \sum_{x_i \in B \cap \mathcal{D}_{test}} \mathbb{I}(y_i = c) \quad (18)$$

where c is the target class, and $\mathbb{I}$ is the trained model.

Instance Coverage:

This quantifies the local breadth of an explanation, defined as the probability of a random sample falling inside box B :

$$\text{Coverage}(B) = P(x \in B) \quad (19)$$

For class-specific analysis, we used *Class-Conditional Coverage*, which measures the fraction of the target class captured by the anchor:

$$\text{Coverage}(B, c) = P(x \in B \mid y = c) \quad (20)$$

6.1.1 Class-Union Metrics

To evaluate the global decision boundary of a class, we analyze the set of all rules $\mathcal{R}_c = \{B_1, \dots, B_k\}$ generated for that class. In the single-agent setting, this set is formed by aggregating independent runs; in the multi-agent setting, this set is the coordinated output of cooperative agents.

Union Coverage:

The primary measure of global completeness is the coverage of the logical disjunction of all rules in $\mathcal{R}_c$. Let $U_c(x)$ be the union mask where $U_c(x) = \bigvee_{B \in \mathcal{R}_c} \mathbb{I}(x \in B)$. Class Union Coverage is

$$\text{Coverage}_{\cup}(c) = P(U_c(x) = 1 \mid y = c) \quad (21)$$

This metric is critical for identifying dataset imbalances. A higher union coverage indicates that the agents have successfully coordinated to cover diverse modes of the class distribution rather than collapsing into a single easy-to-explain region.

Union Precision:

Similarly, Union Precision measures the fidelity of the composite decision boundary formed by the set of rules as follows:

$$\text{Precision}_{\cup}(c) = P(y = c \mid U_c(x) = 1) \quad (22)$$

High union precision requires agents to maintain local precision and avoid *bridging* effects, where overlapping rules might inadvertently capture out-of-distribution regions.

6.1.2 Rule Diversity and Stability

The Multi-Agent framework specifically optimizes these metrics to reduce redundancy and ensure convergence.

Rule Overlap:

To quantify the diversity of the rule set, we computed the pairwise overlap between the rules. For two rules R_1 and R_2 , the overlap is defined as the minimum intersection-over-union (IoU) across all feature dimensions f :

$$\text{Overlap}(R_1, R_2) = \min_{f \in F} \left(\frac{\text{intersection}_f(R_1, R_2)}{\text{union}_f(R_1, R_2)} \right) \quad (23)$$

Because anchors represent logical AND conditions, a geometric overlap occurs only if the rules overlap in *all* dimensions; the minimum operator strictly enforces this geometric constraint. Minimizing this metric ensures that the agents learn distinct and non-redundant explanations.

NashConv:

In the multi-agent setting, we assessed the training stability using NashConv, which is defined as the sum of exploitability across all agents. A low NashConv indicates that the system has converged to an approximate Nash Equilibrium, where no agent can significantly improve its reward by unilaterally deviating, confirming that the learned rule coordination is stable.

6.1.3 Interpretability Metrics

To bridge local rules with global model understanding, we derived feature importance directly from the rule definitions.

Rule-Based Feature Importance:

Unlike permutation importance, which relies on model accuracy drops, we derive importance from the explanatory structure itself. For a feature f , importance is derived from its frequency of occurrence across the rule set and its selectivity (inverse interval width) as follows:

$$\text{Importance}(f) = \frac{\text{Frequency}(f)}{\text{AvgWidth}(f) + \epsilon} \quad (24)$$

where ϵ is a small constant. This metric highlights features that are both frequently relied upon and tightly constrained, identifying the critical decision boundaries of a

Reviewed black-box model.

7 Results

In this section, we define the metrics used and evaluate our proposed dynamic anchor and multiagent frameworks against the baseline.

Reviewed
till
here.
This
is the
older
ver-
sion
of the
results.
Insert
the
new
results
- tak-
ing
rule
and
explain-
ing
here

7.1 Datasets

We evaluate Dynamic Anchors on tabular and numerical classification benchmarks from `scikit-learn`: **Iris**, **Breast Cancer**, **Wine**, and a synthetic non-convex **Circles**. All features are standardized for the classifier, whereas the anchor environment maintains a normalized copy in $[0, 1]^d$ to represent axis-aligned bounds consistently across datasets. Continuous features were normalized for the RL agent but denormalized for human-readable rule extraction. For each dataset, we first trained a base classifier $f : \mathbb{R}^d \rightarrow [0, 1]^K$ and then treated it as a fixed black box during anchor discovery. We report all anchor metrics on a held-out test set $\mathbf{D}_{test}$ to measure the generalization of the explanation rather than memorization of the training data.

7.2 Anchor environment and policy training

Episodes run for at most the default `max_cycles` steps of 100, with optional early termination when updates become stable. To robustly estimate the precision in sparse regions, the environment can generate locally perturbed samples inside the current box. We use an *adaptive* perturbation strategy: if the box covers sufficient real points, we bootstrap from the covered samples; otherwise, we fall back to uniform sampling within the box. Unless otherwise noted, we used a target precision threshold $P_{thr} = 0.95$ and varied the coverage threshold C_{thr} (e.g., 0.1, 0.3 and 0.6) to study the precision–coverage trade-off. We compare: (i) a **Baseline** based on original Anchors, (ii) a **Single-agent** RL policy trained per class, and (iii) A **multi-agent** formulation, where multiple agents may represent each class (`agents_per_class` > 1).

7.3 Results and Analysis

Table 1 reports instance-level precision/coverage for all methods, and class-level precision/coverage plus union metrics for Dynamic Anchors variants.

Baseline: high precision, low coverage, no class-level explanations.

Across all datasets, Baseline achieved strong instance-level precision but very low instance coverage (for example, Iris: coverage ≈ 0.12 – 0.18 ; Wine: ≈ 0.10 – 0.15 ; Breast Cancer: ≈ 0.16 – 0.21). This is the classic Anchors trade-off: rules are conservative and faithful where they apply but explain a small fraction of the data. In addition, the baseline does not directly yield the class-level metrics we target (Table 1 shows “_” entries) because it is designed primarily for instance-level explanations.

Dynamic Anchors improves coverage while maintaining high precision.

On structured tabular datasets (Iris, Wine, Breast Cancer), both Single and Multi substantially increased instance-level coverage while preserving strong instance-level precision.

- **Iris:** Single increases coverage from ~ 0.12 – 0.18 (Baseline) to ~ 0.44 – 0.53 , with precision remaining high. Multi further increases coverage to ~ 0.54 – 0.65 while keeping precision near the baseline range.

Table 1: Per-Dataset precision and coverage metrics

Dataset	C_{thr}	Method	Avg. inst. precision c0	Avg. inst. precision c1	Avg. inst. precision c2	Avg. inst. coverage c0	Avg. inst. coverage c1	Avg. inst. coverage c2	Avg. class precision c0	Avg. class precision c1	Avg. class precision c2	Avg. class coverage c0	Avg. class coverage c1	Avg. class coverage c2	Union Precision	Union coverage	#Rules (unique)
Iris	0.1	Baseline	1.000	0.948	0.956	0.178	0.117	0.169	—	—	—	—	—	—	—	—	—
		Single	0.998	0.812	0.980	0.480	0.530	0.440	1.000	0.833	0.889	0.900	1.000	0.800	0.907 $\pm$ 0.069	0.9 $\pm$ 0.082	27
		Multi	0.992	0.921	0.959	0.540	0.545	0.650	1.000	0.833	0.833	0.900	1.000	1.000	0.889 $\pm$ 0.079	0.967 $\pm$ 0.047	60
	0.3	Baseline	1.000	0.948	0.956	0.178	0.117	0.169	—	—	—	—	—	—	—	—	—
		Single	0.998	0.812	0.980	0.380	0.620	0.360	1.000	1.000	0.875	0.700	1.000	0.700	0.958 $\pm$ 0.059	0.8 $\pm$ 0.141	34
		Multi	0.992	0.921	0.959	0.510	0.540	0.645	1.000	0.833	0.769	1.000	1.000	1.000	0.868 $\pm$ 0.097	1.000	60
wine	0.1	Baseline	0.998	1.000	0.979	0.146	0.098	0.120	—	—	—	—	—	—	—	—	—
		Single	0.996	0.923	0.991	0.271	0.161	0.500	1.000	1.000	1.000	0.667	0.500	1.000	1.000	0.722 $\pm$ 0.208	29
		Multi	0.995	0.958	0.986	0.362	0.325	0.505	1.000	1.000	1.000	1.000	0.786	1.000	1.000	0.929 $\pm$ 0.101	60
	0.3	Baseline	0.998	1.000	0.979	0.146	0.098	0.120	—	—	—	—	—	—	—	—	—
		Single	0.996	0.929	0.990	0.317	0.321	0.505	1.000	1.000	1.000	0.667	0.714	1.000	1.000	0.794 $\pm$ 0.147	37
		Multi	0.994	0.953	0.987	0.362	0.343	0.500	1.000	1.000	1.000	0.917	0.857	1.000	1.000	0.925 $\pm$ 0.059	60
breast_cancer	0.1	Baseline	0.994	0.990	—	0.156	0.205	—	—	—	—	—	—	—	—	—	—
		Single	0.991	0.980	—	0.121	0.101	—	0.897	1.000	—	0.619	0.583	—	0.948 $\pm$ 0.052	0.601 $\pm$ 0.018	25
		Multi	0.999	0.962	—	0.131	0.106	—	1.000	0.981	—	0.714	0.736	—	0.991 $\pm$ 0.009	0.725 $\pm$ 0.011	40
	0.3	Baseline	0.994	0.990	—	0.136	0.205	—	—	—	—	—	—	—	—	—	—
		Single	0.998	0.980	—	0.027	0.037	—	1.000	1.000	—	0.214	0.236	—	1.000	0.225 $\pm$ 0.011	28
		Multi	0.999	0.980	—	0.051	0.051	—	1.000	0.974	—	0.500	0.514	—	0.987 $\pm$ 0.013	0.507 $\pm$ 0.007	40
circles	0.3	Baseline	0.815	0.923	—	0.13	0.228	—	—	—	—	—	—	—	—	—	—
		Single	0.682	0.568	—	0.9	0.604	—	0.5	0.566	—	1.000	0.81	—	0.533 $\pm$ 0.033	0.905 $\pm$ 0.095	21
		Multi	0.999	0.964	—	0.011	0.182	—	1.000	1.000	—	0.19	0.98	—	1.000	0.585 $\pm$ 0.395	33

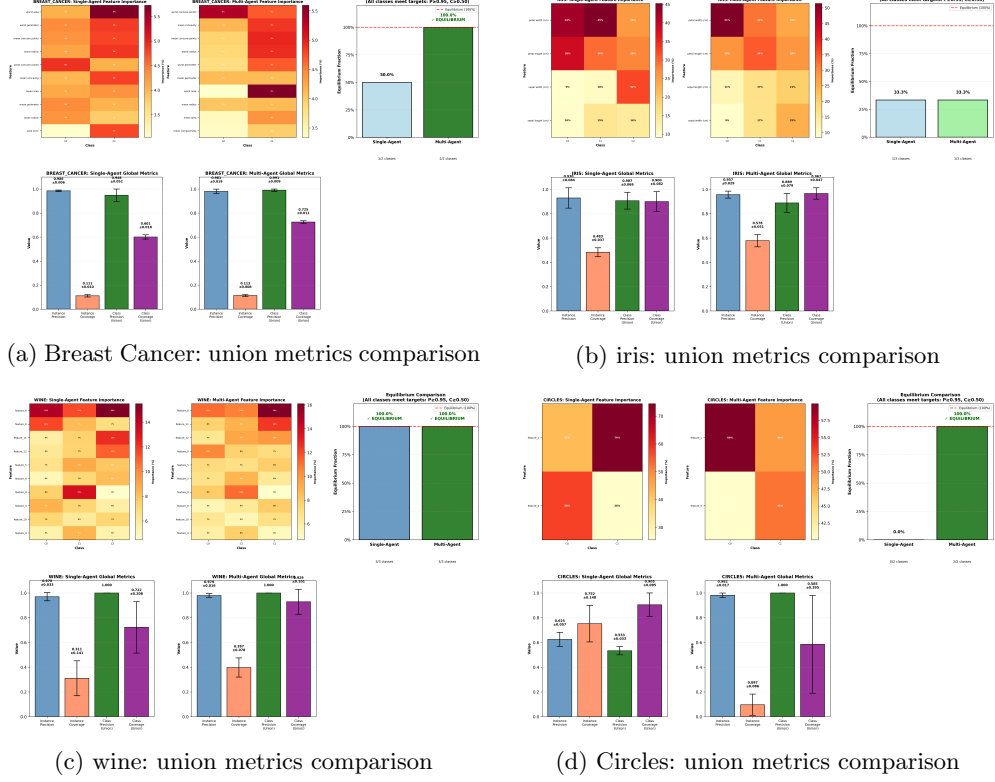


Fig. 2: Union metrics comparison when coverage threshold is 0.1 dataset

- **Wine:** Baseline coverage is ~ 0.10 – 0.15 . Single increases coverage up to ~ 0.50 for some classes, and Multi consistently achieves higher coverage (~ 0.33 – 0.51) at near-perfect precision.
- **Breast Cancer:** The coverage gains are more nuanced (and depend on C_{thr}), but Multi tends to dominate Single in union coverage while maintaining very high union precision.

Multi-agent yields the best union coverage (global completeness).

The key advantage of Multi is visible in the **union** metrics (last columns of Table 1). When multiple rules are learned per class, the union of those rules tends to cover a larger fraction of that class while remaining faithful.

- **Iris** ($C_{thr} = 0.1$): union coverage improves from 0.90 (Single) to 0.967 (Multi), with a modest decrease in union precision ($0.907 \rightarrow 0.889$), indicating Multi trades a small amount of fidelity for substantially higher completeness.
- **Wine** ($C_{thr} = 0.1, 0.3$): union precision is 1.0 for both Single and Multi, but Multi improves union coverage substantially (e.g., $0.722 \rightarrow 0.929$ at $C_{thr} = 0.1$), showing Multi adds complementary rules rather than merely duplicating the same region.

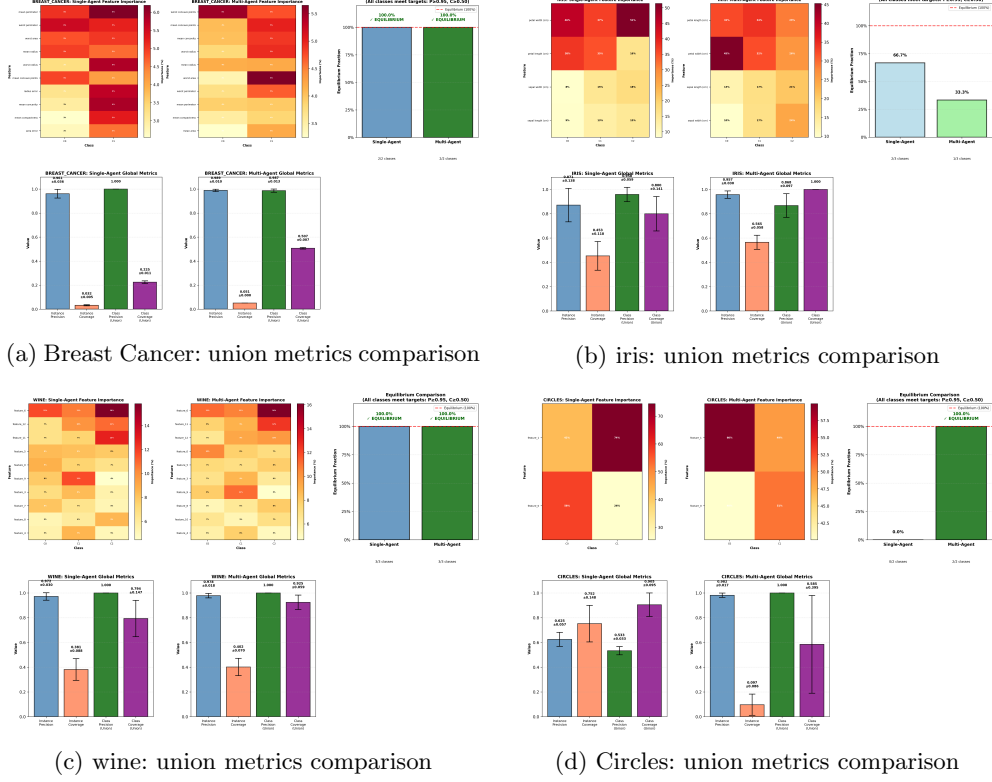


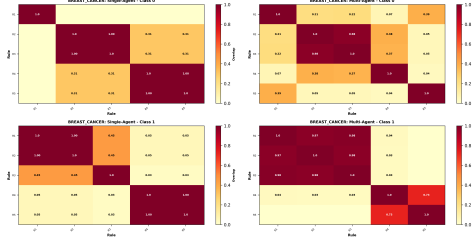
Fig. 3: Union metrics comparison when coverage threshold is 0.3 dataset

- **Breast Cancer:** Multi improves union coverage over Single in both settings (e.g., $0.601 \rightarrow 0.725$ at $C_{thr} = 0.1$; and $0.225 \rightarrow 0.507$ at $C_{thr} = 0.3$), while keeping union precision very high (≈ 0.99).

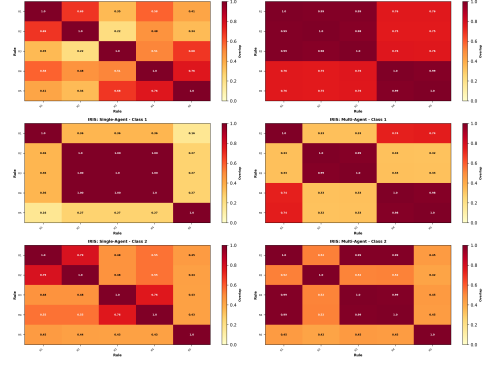
This supports the intended role of MARL: multiple agents can partition (or partially partition) the explanatory space, improving class completeness without reducing precision.

Rule budget: Multi produces more rules, improving completeness but increasing complexity.

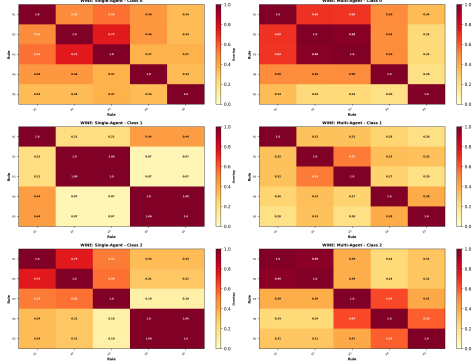
Multi consistently produced more unique rules than Single (for example, Iris/Wine: 60 vs. 27–37; Breast Cancer: 40 vs. 25–28; Circles: 33 vs. 21). This is expected because Multi uses multiple agents per class; the empirical takeaway is that **the added rule budget translates into higher union coverage**, particularly for Iris/Wine/Breast Cancer. In applications where interpretability requires a hard rule budget, this trade-off can be controlled by the number of agents per class and rule-filtering thresholds.



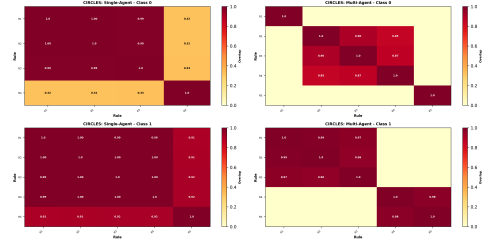
(a) Breast Cancer: rule overlap matrix



(b) iris: rule overlap matrix



(c) wine: rule overlap matrix

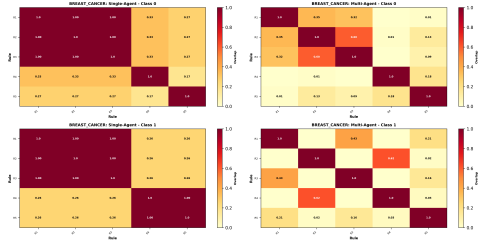


(d) circles: rule overlap matrix

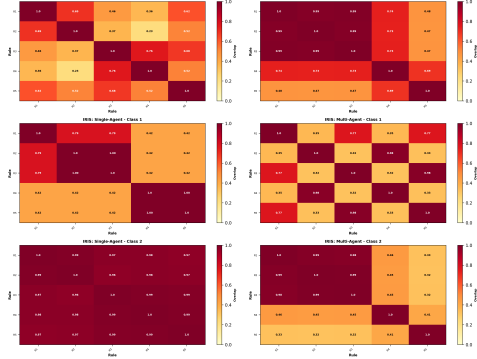
Fig. 4: Rule overlap comparison when coverage threshold is 0.1 dataset

7.4 Effect of the coverage threshold C_{thr}

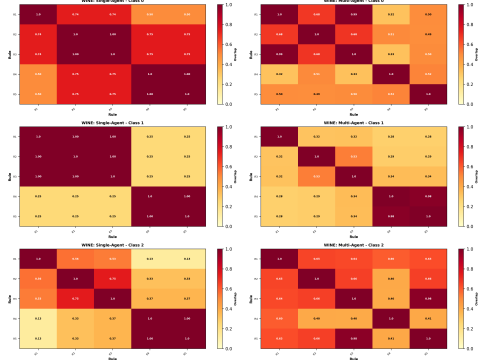
Varying C_{thr} exposes the precision–coverage tension differently across datasets. On Iris and Wine, increasing C_{thr} generally maintains high precision and can push Multi toward higher union coverage (e.g., Iris Multi reaches union coverage 1.0 at $C_{thr} = 0.3$). However, for Breast Cancer, the instance-level coverage of Single drops sharply at $C_{thr} = 0.3$ (suggesting that forcing broader or more stable rules can be difficult under the current reward/termination dynamics), while Multi remains substantially more complete at the union level. Overall, C_{thr} acts as a meaningful knob: it can bias the learned rules toward broader applicability but may also increase training difficulty on higher-dimensional or imbalanced datasets.



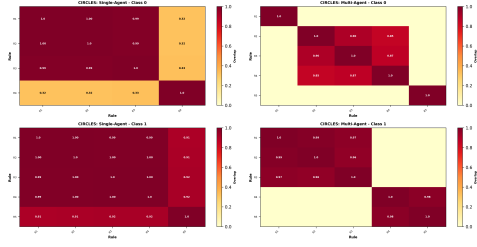
(a) Breast Cancer: rule overlap matrix



(b) iris: rule overlap matrix



(c) wine: rule overlap matrix



(d) Circles: rule overlap matrix

Fig. 5: Rule overlap comparison when coverage threshold is 0.3 dataset

7.5 Qualitative analysis with plots

Precision–coverage comparisons mirror the table and highlight systematic gains.

The per-dataset precision/coverage plots (Fig. 3b, Fig. 2c, Fig. 3a) shows a consistent Baseline achieves strong precision but is conservative in coverage, Single expands coverage substantially, Multi further increases coverage—especially when unioned—while retaining high precision on the structured datasets. These plots make the central claim visually clear: **Dynamic Anchors shifts the Pareto frontier outward.**

Rule overlap heatmaps diagnose redundancy vs. complementarity.

The rule overlap heatmaps (Fig. 4a) reveals the extent to which different extracted rules cover the same region. A high overlap indicates redundancy (multiple rules describing essentially the same subregion), whereas a low overlap suggests complementary decomposition of the class. In several cases, Multi increases union coverage even

when some overlap exists, implying that at least a subset of rules is meaningfully complementary. This supports the use of overlap penalties/regularizers, which are useful not only for avoiding inter-class conflicts but also for encouraging intra-class diversity to improve coverage efficiently.

Feature-usage summaries provide a global explanation narrative.

Comprehensive panels (for example, see Fig. Figure 3b) summarizes the features that are repeatedly constrained across the extracted rules, yielding an intuitive global story: for Iris, petal-related features dominate the rule structure (consistent with domain expectations), Sepal features played a smaller role. These summaries complement precision/coverage metrics by indicating *what the rules rely on*, which is useful for communicating explanations to practitioners.

7.6 Rule example and analysis

In this section, we provide a detailed explanation of a few example rules produced by the framework.

7.7 Failure modes and hard cases: Circles

The Circles dataset is intentionally non-convex; axis-aligned boxes cannot represent ring-shaped decision regions without either (i) covering large irrelevant areas (hurting precision) or (ii) shrinking to small local fragments (hurting coverage). This tension is evident in Table 1. Single attains high union coverage (≈ 0.905) but low union precision (≈ 0.533), which is consistent with overly broad boxes. Multi achieves very high union precision (1.0) but unstable/variable union coverage (0.585 ± 0.395), suggesting that agents often converge to small high-precision fragments that do not collectively cover the full ring. This result is important because it clearly exposes the representational limitations of box rules on non-convex manifolds. In practice, improving Circles-like performance would require either richer rule primitives (e.g., unions of multiple boxes per agent, oblique splits, or prototype-based regions) or curriculum strategies that encourage coverage expansion without collapsing precision.

7.8 Key takeaways

- **Dynamic Anchors improves completeness:** Compared to static Anchors, RL-based refinement substantially increases coverage while retaining high precision on structured tabular datasets.
- **Multi-agent improves global class explanations:** Multi yields the strongest union coverage across Iris/Wine/Breast Cancer, demonstrating the benefit of multiple complementary rules per class.
- **Complexity-completeness trade-off:** Multi produces more rules; this improves completeness but increases explanation size.
- **Geometry matters:** Non-convex decision regions (Circles) expose limitations of axis-aligned boxes; results motivate extending the rule language or adding stronger diversity/coverage shaping.

8 Discussion and Limitations

9 Conclusion and Future work

In conclusion, dynamic anchors produce a reusable policy that can help explain both instance- and class-level human-readable explanations. For simpler datasets, a single agent can provide good explanations. The multi-agent setup provides an advantage by generating more unique rules with less overlap, which can help cover the samples in sparser regions. Several directions are available for future studies. The current study only covers tabular data and can be expanded to other data modalities. The concept of equilibrium in the current study is ill-defined; thus, we do not reach a true Nash equilibrium with the multi-agent setup.

References

- [1] Molnar, C., Casalicchio, G., Bischl, B.: Interpretable machine learning – a brief history, state-of-the-art and challenges. In: Koprinska, I., Kamp, M., Appice, A., Loglisci, C., Antonie, L., Zimmermann, A., Guidotti, R., Ozgobek, O., Ribeiro, R.P., Gavalda, R., Gama, J., Adilova, L., Krishnamurthy, Y., Ferreira, P.M., Malerba, D., Medeiros, I., Ceci, M., Manco, G., Masciari, E., Ras, Z.W., Christen, P., Ntoutsi, E., Schubert, E., Zimek, A., Monreale, A., Biecek, P., Rinzivillo, S., Kille, B., Lommatzsch, A., Gulla, J.A. (eds.) ECML PKDD 2020 Workshops vol. 1323, pp. 417–431. Springer. https://doi.org/10.1007/978-3-030-65965-3_28 . Series Title: Communications in Computer and Information Science. https://link.springer.com/10.1007/978-3-030-65965-3_28 Accessed 2025-12-17
- [2] Lundberg, S., Lee, S.-I.: A Unified Approach to Interpreting Model Predictions. arXiv. <https://doi.org/10.48550/arXiv.1705.07874> . <http://arxiv.org/abs/1705.07874> Accessed 2025-12-17
- [3] Ribeiro, M.T., Singh, S., Guestrin, C.: "Why Should I Trust You?": Explaining the Predictions of Any Classifier. arXiv. <https://doi.org/10.48550/arXiv.1602.04938> . <http://arxiv.org/abs/1602.04938> Accessed 2025-12-17
- [4] Ribeiro, M.T., Singh, S., Guestrin, C.: Anchors: High-precision model-agnostic explanations **32**(1) <https://doi.org/10.1609/aaai.v32i1.11491> . Accessed 2025-12-11
- [5] Clark, P., Niblett, T.: The CN2 induction algorithm **3**(4), 261–283 <https://doi.org/10.1007/BF00116835> . Accessed 2025-12-11
- [6] Cohen, W.W.: Fast effective rule induction. In: Machine Learning Proceedings 1995, pp. 115–123. Elsevier. <https://doi.org/10.1016/B978-1-55860-377-6.50023-2> . <https://linkinghub.elsevier.com/retrieve/pii/B9781558603776500232> Accessed 2025-12-11
- [7] Kotschieder, P., Fiterau, M., Criminisi, A., Buló, S.R.: Deep neural decision

- forests. In: 2015 IEEE International Conference on Computer Vision (ICCV), pp. 1467–1475. IEEE. <https://doi.org/10.1109/ICCV.2015.172> . <http://ieeexplore.ieee.org/document/7410529/> Accessed 2025-12-17
- [8] Yang, Y., Morillo, I.G., Hospedales, T.M.: Deep Neural Decision Trees. arXiv. <https://doi.org/10.48550/arXiv.1806.06988> . <http://arxiv.org/abs/1806.06988> Accessed 2025-12-17
 - [9] Roth, A.M., Topin, N., Jamshidi, P., Veloso, M.: Conservative Q-Improvement: Reinforcement Learning for an Interpretable Decision-Tree Policy. arXiv. <https://doi.org/10.48550/arXiv.1907.01180> . <http://arxiv.org/abs/1907.01180> Accessed 2025-12-11
 - [10] Bastani, O., Pu, Y., Solar-Lezama, A.: Verifiable Reinforcement Learning via Policy Extraction. arXiv. <https://doi.org/10.48550/arXiv.1805.08328> . <http://arxiv.org/abs/1805.08328> Accessed 2025-12-17
 - [11] Heinrich, J., Heinrich, J., Lanctot, M.: Paper 3: Fictitious self-play in extensive-form games
 - [12] Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., Mordatch, I.: Paper 4: Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. arXiv. <https://doi.org/10.48550/arXiv.1706.02275> . <http://arxiv.org/abs/1706.02275> Accessed 2025-11-03
 - [13] Pu, Y., Wang, S., Yang, R., Yao, X., Li, B.: Decomposed Soft Actor-Critic Method for Cooperative Multi-Agent Reinforcement Learning. arXiv. <https://doi.org/10.48550/arXiv.2104.06655> . <http://arxiv.org/abs/2104.06655> Accessed 2025-12-18
 - [14] Bettini, M., Prorok, A., Moens, V.: BenchMARL: Benchmarking Multi-Agent Reinforcement Learning. arXiv. <https://doi.org/10.48550/arXiv.2312.01472> . <http://arxiv.org/abs/2312.01472> Accessed 2025-12-11
 - [15] Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., Dormann, N.: Stable-baselines3: Reliable reinforcement learning implementations
 - [16] Terry, J.K., Black, B., Grammel, N., Jayakumar, M., Hari, A., Sullivan, R., Santos, L., Perez, R., Horsch, C., Dieffendahl, C., Williams, N.L., Lokesh, Y., Ravi, P.: PettingZoo: Gym for Multi-Agent Reinforcement Learning. arXiv. <https://doi.org/10.48550/arXiv.2009.14471> . <http://arxiv.org/abs/2009.14471> Accessed 2025-12-11
 - [17] Lanctot, M., Zambaldi, V., Gruslys, A., Lazaridou, A., Tuyls, K., Perolat, J., Silver, D., Graepel, T.: A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning. arXiv. <https://doi.org/10.48550/arXiv.1711.00832> . <http://arxiv.org/abs/1711.00832> Accessed 2025-12-18

Appendix A Section title of first appendix

The appendix contains supplementary information that is not an essential part of the text itself but may be helpful in providing a more comprehensive understanding of the research problem or information that is too cumbersome to be included in the body of the paper.